

Letta — Autonomous Invoice-Factoring Agent on Arc

Ignyte Stablecoins Commerce Stack Challenge · Track 2 — SME Trade Finance & Working Capital

Project Title	Letta — an autonomous invoice-factoring agent on Arc, settled in USDC
Track	Track 2 — SME Trade Finance & Working Capital (invoice factoring with repayment waterfalls)
Circle Account Email	wangligang16161616@gmail.com
GitHub Repository	https://github.com/wanggang22/arcpay/tree/letta
Video Demo	(88s HyperFrames walkthrough — link to be added on upload)

Problem

SMEs die on cash flow. They ship goods, issue an invoice with 30–90 day terms, then wait months for money they need *now* to make payroll and buy materials. Factoring fixes this — a financier advances cash against the invoice and collects from the buyer later — but underwriting (should we advance? how much? at what discount?) is slow, manual, and closed to small suppliers. There is no rail to do it programmatically in stablecoins.

Description

Letta is an AI agent that is a **credit committee with money at stake**. It reads the buyer's **real on-chain payment history on Arc**, prices an advance (advance % + discount rate) with a written reasoning memo, disburses USDC to the supplier now, and runs a **3-way repayment waterfall** as the buyer pays — recovering the factor's advance + fee first, surplus to the supplier. Partial and late payments are first-class: the agent **re-assesses exposure live** on every repayment.

The autonomy is genuine because the decision is a judgment with real money at risk, not "approve a document I wrote" — the failure mode that makes most agentic-finance demos read as theater. Built on Arc: native-USDC settlement, sub-second finality. It reuses ArcPay's PayPerCall `Paid` events as the buyer's unfakeable credit signal, and Obol's decide→pay→verify agent loop, repointed from "which service to buy" to "should I advance against this invoice and at what price."

Products Used (Circle / Arc)

- **USDC** — native gas, every payout is `pay{value}` ; no ERC-20 approval dance.
- **Arc smart contracts** — `FactoringPool.sol` deployed on Arc testnet; the buyer's credit signal is on-chain `Paid` receipts from ArcPay's PayPerCall.

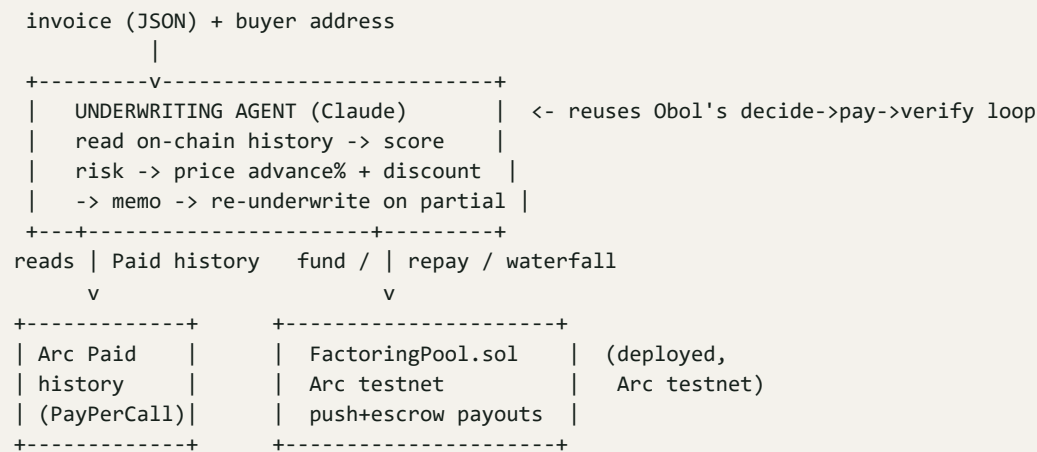
- **Arc sub-second finality** — fund, partial, and settle all clear in under half a second.
- **Anthropic Claude** — the underwriter.
- Roadmap: Circle Wallets (agent custody), Circle Gateway + CCTP (cross-chain advance capital), USYC (idle-float yield as the factor's funding source).

Working MVP

```
cd letta && npm install
npm test           # 14 unit tests (features, underwriting, waterfall math)
npm run typecheck
npx tsx scripts/live.ts # REAL end-to-end on Arc testnet
```

Contract: `FactoringPool.sol` (forge, 18 tests). Agent: `history.ts` (credit features from on-chain Paid events), `underwrite.ts` (Claude prices advance% + discount + memo, deterministic fallback), `pool.ts` / `orchestrator.ts` (fund + live exposure re-assessment).

Architecture



Push-with-escrow-fallback payouts: a grieving recipient can never brick the waterfall or strand the factor's recovery (a 5-lens adversarial security review found + fixed a real push-payment DoS and an overpayment leak before deploy).

Traction — verified live on Arc testnet, in-window

Claude underwrote a thin-file buyer and the contract settled the full waterfall with real test-USDC:

1. **Underwrite [Claude]:** advance 70% (700,000 wei), discount 4.5% (fee 45,000 wei). Memo: *"Only 3 prior payments — a very thin file — with 0.67 counterparty concentration; 210 wei of history vs a 1,000,000 wei invoice is a size mismatch; recent activity encouraging but sparse history justifies a moderate advance."*
2. **Fund:** tx `0xbbf25219d092b43dfa6223971e714eba31fc06d6fdc2dfe4df72d7c3727771c4` — supplier +700,000 wei.
3. **Partial (40%):** tx `0xebb83f06084df34aed7e7a65bd7b6e92d3d64bbf3ca18a570e98798eb38aa6b1` — all to factor; agent re-rated exposure 745k → 345k.

4. **Late remainder + settle:** tx `0x764240eaf5aff49af0d5d525d19e7585e63029f7c7591f0bca8e77c98793c9d2` — factor whole at 745k, supplier surplus 255k, settled.

Outcome: collected 1,000,000 · factor recovered 745,000 (profit = fee 45,000) · supplier 955,000 (= face – fee). FactoringPool: `0xcE939A8048b8BF5bE9DfE639d4C65227A99901F5`

Capital model — the operator fronts no money (LP-funded pool)

Real invoice-financing platforms don't lend their own balance sheet — investors fund the advances and the platform underwrites. `LettaPool.sol` (`0x66a8fd4cd737dC91ca285c6eC7Ef798cFa9Af617`) is that: LPs `deposit()` USDC for shares (a claim on idle cash + outstanding receivables), the agent funds advances **from the pool**, and on repayment the principal returns while the **fee accrues as LP yield** (share price rises). **Proven live on Arc testnet:** an LP deposited 3,000,000 wei → the agent funded a 700,000 advance from the pool → the buyer repaid → the pool grew to 3,045,000 (the 45,000 fee = LP yield) → the LP withdrew. 9 forge tests cover the share math, fund-from-pool, principal+fee recovery, pro-rata yield, and liquidity limits. So the operator just runs the underwriting engine and never fronts capital; buyers and suppliers bring their own wallets.

Onboarding — Circle Wallets (email login, no MetaMask)

SMEs aren't crypto-native, so the product onboards with **Circle Wallets (User-Controlled, email/PIN)**: a user signs in with an email and gets an **Arc wallet — no MetaMask, no seed phrase**. The backend (`src/circle.ts`, `/api/circle/*`) is **verified live on ARC-TESTNET**: create user (201) → session token (200) → wallet-init challenge (201, challenge issued); Circle lists `ARC-TESTNET` as a supported chain. The Web SDK PIN modal is wired for the production (Vite/Next) frontend.

Interactive console — working frontend + backend

`npm run app` → `localhost:8088` serves a guided console over the live pool: deposit as an LP, then establish buyer history → underwrite (Claude prices it) → fund from the pool → buyer pays 40% (pool recovers principal) → late remainder (settles, fee → LP yield) → LP withdraws with yield. Every step is a real Arc transaction with an arcscan link. Verified end-to-end by clicking through the UI.

Documentation

In the repo (`letta/` on the `letta` branch): `SUBMISSION.md`, `README.md` (run instructions), `docs/superpowers/specs/2026-06-23-letta-factoring-agent-design.md` (design), `evidence/run.json` (live-run evidence), `dashboard/index.html` (visual), `contracts/src/FactoringPool.sol` + tests. MIT licensed.

Product Feedback (Circle / Arc)

What worked: Arc's native-USDC gas made the contract trivial — payouts are just `pay{value}`, no token approvals. Sub-second finality made fund → partial → settle feel instantaneous, which is what sells an agentic finance demo. Reusing on-chain `Paid` events as a credit signal is a clean, unfakeable data source no off-chain underwriter has.

Where to improve: (1) the testnet RPC caps `eth_getLogs` to a 10k-block range with no pagination cursor — reading an address's full history needs manual chunking; a richer logs/indexer endpoint would help agents that reason over history. (2) Native-USDC-as-gas with 18-decimal wei is elegant but surprising (USDC is usually 6-decimal) — a docs note on the decimals model would prevent confusion. (3) More worked "agentic finance on Arc" examples (escrow, factoring, payroll) would shortcut a lot of first-principles contract work.